

Deep Learning Fundamentals Mid 2 Solution

Module 4: Model Evaluation

1. Explain the importance of model evaluation in deep learning.
 1. **Measuring Model Performance:** Model evaluation helps **determine if a model is effective** at performing its task using performance metrics such as accuracy, precision, recall, F1-score, or loss values.
 2. **Detecting Overfitting and Underfitting:** Evaluation allows us to **compare training and testing performance**.
 1. **Overfitting:** The model performs well on training data but poorly on unseen data.
 2. **Underfitting:** The model performs poorly on both training and test data due to insufficient learning.By analyzing evaluation results, developers can adjust the model architecture or training process to **improve generalization**.
 3. **Model Comparison and Selection:** Multiple models or configurations can be trained for the same problem. Evaluation metrics help compare these models objectively and **select the best-performing one**.
 4. **Hyperparameter Tuning:** Model evaluation is **essential when tuning hyperparameters** such as learning rate, batch size, or network depth.

2. Describe commonly used performance metrics for classification problems.

1. **Accuracy:** Measures the proportion of correctly predicted instances out of the total instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

It is useful when the dataset is balanced, but it may be misleading for imbalanced datasets.

2. **Precision:** Measures the proportion of correctly predicted positive instances among all instances predicted as positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision means the model produces fewer false positives.

3. **Recall (Sensitivity):** Measures the proportion of actual positive instances that are correctly identified by the model.

$$\text{Recall} = \frac{TP}{TP + FN}$$

High recall indicates that the model detects most of the positive cases.

4. **F1-Score:** The harmonic mean of precision and recall. It provides a balance between the two metrics.

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

It is particularly useful when there is an uneven class distribution.

5. **Specificity (True Negative Rate):** Measures the proportion of actual negative cases that are correctly classified.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

It indicates how well the model avoids false alarms.

3. Explain the purpose of baseline models.

A **baseline model** is a simple model used as a **reference point to evaluate more complex machine learning or deep learning models**. It provides a minimum performance benchmark that improved models should be able to surpass. Baseline models help **measure whether new models actually improve performance**, simplify early development, and detect potential issues in data or model design. They also **guide further experimentation and optimization** by providing a standard for comparison.

4. Analyze the advantages and limitations of grid search.

Advantages of Grid Search

1. **Systematic Exploration:** Grid search evaluates every combination of specified hyperparameters. This ensures a thorough and structured search of the parameter space.
2. **Easy to Implement and Understand:** It is simple to use and widely supported in machine learning libraries. The concept is straightforward because it tests all combinations in a predefined grid.
3. **Reproducible Results:** Since the search space is predefined and deterministic, grid search produces consistent and reproducible results across runs.
4. **Suitable for Small Parameter Spaces:** When the number of hyperparameters and possible values is small, grid search can effectively identify the optimal configuration.

Limitations of Grid Search

1. **High Computational Cost:** Grid search becomes very expensive when the number of hyperparameters or their possible values increases. The total combinations grow exponentially.
2. **Inefficient Search:** Many combinations tested may be unnecessary or irrelevant, which wastes computation time.
3. **Limited Coverage of Parameter Space:** Grid search only evaluates values specified in the grid. If the optimal value lies between two grid points, it will not be discovered.
4. **Not Scalable for Deep Learning Models:** Deep learning models often have many hyperparameters and require long training times, making grid search impractical for large-scale problems.

5. Evaluate when random search is preferred over grid search.

Random search is preferred over grid search in the following situations:

1. **Large Hyperparameter Space:** When there are many hyperparameters or many possible values, grid search becomes computationally expensive, while random search samples fewer combinations.
2. **Limited Computational Resources:** Random search requires less time and computing power because it does not test every possible combination.
3. **Continuous Parameter Ranges:** For parameters like learning rate that have continuous values, random search can sample directly from the range.
4. **Unequal Parameter Importance:** If only a few hyperparameters strongly affect performance, random search is more likely to explore useful values.

6. Given $TP = 50$, $FP = 10$, $FN = 20$, $TN = 120$, compute accuracy, precision, and recall.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{50+120}{50+120+10+20} = \frac{170}{200}$$

$$\text{Accuracy} = 0.85 = 85\%$$

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{50}{50+10} = \frac{50}{60}$$

$$\text{Precision} = 0.8333 = 83\%$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{50}{50+20} = \frac{50}{70}$$

$$\text{Recall} = 0.7143 = 71.43\%$$

7. Compute RMSE for actual values $\{2, 4, 6\}$ and predicted values $\{3, 5, 5\}$.

Step 1: Calculating Errors

- $(3 - 2) = 1$
- $(5 - 4) = 1$
- $(5 - 6) = -1$

Step 2: Calculating Squared Errors

- $1^2 = 1$
- $1^2 = 1$
- $-1^2 = 1$

Step 3: Calculating Mean Squared Error (MSE)

$$\text{MSE} = \frac{1+1+1}{3} = \frac{3}{3} = 1$$

Step 4: Calculating Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{1} = 1$$

Thus the RMSE for the given values is 1.

8. Analyze two models using confusion matrices and identify the better one for an imbalanced dataset.

For **imbalanced datasets**, accuracy alone is not reliable because the majority class can dominate the result. Instead, metrics such as precision, recall, and F1-score should be analyzed from the confusion matrix.

Steps to Compare Two Models:

1. **Examine the Confusion Matrices:** Each confusion matrix provides **TP** (True Positives), **FP** (False Positives), **FN** (False Negatives), and **TN** (True Negatives)
2. **Calculate key metrics such as precision, recall, and F1 score.**
3. **Focus on Recall and F1-score:** For imbalanced datasets, correctly identifying the minority class is important. **A model with higher recall and F1-score for the minority class is generally preferred.**

Decision Rule:

- If Model A has higher accuracy but low recall for the minority class, it may be misleading.
 - If Model B has higher recall and F1-score, it is usually the better model for imbalanced data.
9. Compare computational cost of grid search with 5 hyperparameters vs random search with 10 trials.

Grid Search

- If each of the 5 hyperparameters has k possible values, **total evaluations** = k^5
- **Example:** If each hyperparameter has 5 values, there will be $5^5 = 3125$ model trainings.

Random Search

- For a random search with 10 trials, **total evaluations** = 10

Thus, grid search has **exponentially higher computational cost as hyperparameters increase**, while random search keeps the cost fixed. Therefore, **random search is much more computationally efficient for large hyperparameter spaces.**

10. Evaluate whether accuracy is a reliable metric for a dataset with 95% negative samples.

Accuracy is not a reliable metric for a dataset with 95% negative samples because the data is highly imbalanced. A model could simply predict all samples as negative and still achieve about 95% accuracy while completely failing to detect positive cases. This makes accuracy misleading since it does not reflect performance on the minority class. For such datasets, metrics like precision, recall, and F1-score are more appropriate because they better evaluate how well the model identifies the minority class.

Module 5: Auto-encoders and Generative Models

1. Explain the concept of undercomplete autoencoders.

An **undercomplete autoencoder** is a type of autoencoder where the **hidden layer (latent representation)** has fewer neurons than the input layer. This forces the network to compress the input data into a smaller representation before reconstructing it at the output.

Because the latent space is smaller, the model cannot simply copy the input directly. Instead, it must learn the most important features or patterns in the data to recreate the input as accurately as possible. This makes undercomplete autoencoders **useful for feature extraction, dimensionality reduction, and data compression** while preserving essential information.

2. Describe regularized autoencoders.

Regularized autoencoders are autoencoders that **include additional constraints or penalties during training** to prevent the model from simply copying the input to the output. These regularization techniques **force the network to learn more meaningful and robust feature representations**.

By restricting the model's capacity, **regularized autoencoders improve generalization and feature learning**, making them useful for tasks like representation learning and dimensionality reduction.

3. Explain how variational autoencoders generate new samples.

Variational Autoencoders (VAEs) generate new samples by learning a probability distribution of the training data rather than a fixed encoding.

During training, the encoder maps each input into parameters of a probability distribution in the latent space. Instead of producing a single latent vector, the model samples a point from this distribution.

The decoder then takes this sampled latent vector and reconstructs it into data similar to the original input. After training, new samples can be generated by randomly sampling points from the latent distribution and passing them through the decoder, which produces new data resembling the training dataset.

4. Analyze the training challenges of GANs.

The challenges of Generative Adversarial Networks (GANs) include:

- **Training Instability:** The generator and discriminator are trained simultaneously in a competitive process, which can cause oscillations and difficulty in reaching convergence.
- **Mode Collapse:** The generator may produce only a small variety of outputs, failing to capture the full diversity of the data distribution.
- **Vanishing Gradients:** If the discriminator becomes too strong, the generator receives very small gradients and cannot learn effectively.
- **Hyperparameter Sensitivity:** GAN training is sensitive to choices like learning rate, batch size, and network architecture.
- **Balance Between Networks:** The generator and discriminator must learn at similar rates; if one becomes much stronger than the other, training becomes ineffective.

5. Evaluate the usefulness of autoencoders in representation learning.

Autoencoders are useful in representation learning because they **automatically learn compact latent features from raw data without requiring labeled datasets or manual feature engineering**. By compressing high dimensional data into a lower dimensional representation, **they capture important patterns and structures in the data**, making them effective for tasks like dimensionality reduction and feature extraction. **However, without proper constraints or regularization, autoencoders may simply learn to copy the input rather than meaningful representations.**

6. An autoencoder has input size 512 and bottleneck size 64. Compute the compression ratio.

Compression ratio is defined as:

$$\text{Compression Ratio} = \text{Input Size} / \text{Bottleneck Size}$$

Given,

- Input size = 512
- Bottleneck size = 64

$$\text{Compression Ratio} = 512 / 64 = 8$$

Thus, the compression ratio = 8:1, meaning the representation is compressed 8 times smaller than the input.

7. Given reconstruction errors across epochs, determine whether learning is improving.

Learning is improving if the reconstruction error decreases over epochs. A steady reduction in error indicates that the autoencoder is learning better representations and reconstructing the input more accurately. **If the error stays constant, the model may not be learning effectively, and if it increases, the training process may be unstable.** Therefore, a consistent downward trend in reconstruction error suggests that learning is improving.

8. Compare reconstruction loss values of undercomplete and regularized autoencoders.

Undercomplete autoencoders typically aim to minimize reconstruction loss by compressing the input into a smaller latent space, so their reconstruction loss is often relatively low if the bottleneck still captures essential information. However, the compression constraint may sometimes slightly increase the loss compared to unconstrained models.

Regularized autoencoders include additional constraints such as sparsity penalties or noise. Because of these constraints, they often show higher reconstruction loss during training, but they learn more robust and meaningful representations and generalize better to new data.

In comparison, **undercomplete autoencoders usually achieve lower reconstruction loss**, while **regularized autoencoders may have slightly higher loss but better feature learning and generalization**.

9. Analyze generator and discriminator loss curves to identify GAN instability.

By examining the generator and discriminator loss curves during training, we can identify the GAN instability with the following cases:

- **Stable training:** Both generator and discriminator losses fluctuate but remain within a reasonable range. Neither network consistently dominates the other.
- **Discriminator loss near zero:** Indicates the discriminator is too strong, easily distinguishing real and fake samples, which can cause vanishing gradients for the generator.
- **Generator loss increasing rapidly:** Suggests the generator is failing to improve and cannot fool the discriminator.
- **Large oscillations in both losses:** Indicates unstable training where the networks keep overpowering each other without converging.
- **Sudden drop in generator diversity with low loss:** May indicate mode collapse, where the generator produces limited outputs.

10. Given KL divergence and reconstruction error values, evaluate the performance of a VAE.

To evaluate a Variational Autoencoder (VAE), both reconstruction error and KL divergence are considered because the VAE loss combines these two terms.

- **Low reconstruction error** indicates that the **decoder can accurately reconstruct the input data from the latent representation**.
- **Low or moderate KL divergence** shows that the **learned latent distribution is close to the desired prior distribution** (usually a standard normal distribution).

Evaluation:

- **Low reconstruction error + reasonable KL divergence:** Good performance. The VAE reconstructs data well while maintaining a well-structured latent space.
- **Low reconstruction error + very high KL divergence:** The model reconstructs well but the latent space may not follow the desired distribution, reducing generative quality.
- **High reconstruction error + low KL divergence:** The latent space is well regularized, but the model is not reconstructing data effectively.
- **High values for both:** Poor model performance.